

Yesterday 9:14 PM

why preflight is displayed rarely. not able to see with multiple services

If you're talking about **CORS preflight (OPTIONS) requests**, the reason you see them ^{only} rarely—especially when calling multiple services—is that **the browser does not send a preflight for every API request**. Share ...

1. When does the browser send preflight?

A browser sends an **OPTIONS** preflight before the actual request when the request is considered **non-simple**.

For example:

```
</> http

OPTIONS /api/orders
Origin: https://frontend.example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization, content-type
```

If the server responds successfully:

```
</> http

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://frontend.example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: authorization, content-type
```

then the browser sends:

```
</> http

POST /api/orders
```

2. Why don't you see it for every service?

There are several reasons.

Case A — The request is "simple"

A request such as:

```
</> http
GET /api/products
```



may not require preflight if it satisfies the CORS "simple request" conditions.

So you may see:

```
GET /api/products
```



but no:

```
OPTIONS /api/products
```



Case B — Browser caches the preflight

This is probably the **most common reason** you're seeing it rarely.

Suppose your Angular application calls:

```
Service A → https://api.company.com/users
Service B → https://api.company.com/orders
Service C → https://api.company.com/products
```



The browser may perform:

```
OPTIONS /users
  ↓
OPTIONS successful
  ↓
GET /users
```



But after the CORS permission is cached, subsequent requests may go directly:

```
GET /users
GET /users
GET /users
```



without another `OPTIONS` .

The server can control how long the preflight result can be cached using:

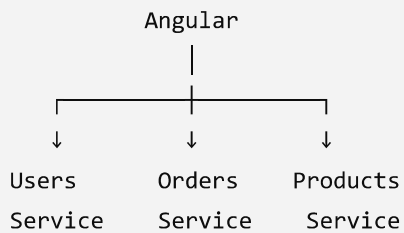
```
</> http
```

```
Access-Control-Max-Age: 3600
```

So preflight is not necessarily one `OPTIONS` per API call.

3. Multiple services don't necessarily mean multiple preflights

Imagine:



You might expect:

```
OPTIONS → Users  
OPTIONS → Orders  
OPTIONS → Products
```

But the browser's CORS behavior depends on things like:

- origin
- HTTP method
- request headers
- credentials mode
- URL/origin
- whether the preflight result is cached

Therefore you might see:

```
OPTIONS /users  
GET    /users  
  
GET    /orders  
  
GET    /products
```

POST /orders

rather than an OPTIONS request before every API call.

4. A very important point: CORS is based on origin

Suppose your Angular application runs at:

`https://app.company.com`



and your services are:

`https://users.company.com`
`https://orders.company.com`
`https://products.company.com`



These are **different origins** because the hostnames differ.

`app.company.com`
↓
`users.company.com` ← different origin
`orders.company.com` ← different origin
`products.company.com` ← different origin



So CORS applies to each cross-origin request.

But that still **doesn't mean OPTIONS must appear for every request.**

5. Headers are often what trigger preflight

For example, Angular commonly sends:

```
</> http

Authorization: Bearer xxx
Content-Type: application/json
```



An API request like:

```
</> http
```



```
POST /orders
```

```
Authorization: Bearer xxx
```

```
Content-Type: application/json
```

will commonly require preflight.

The browser effectively asks:

```
</> http
```

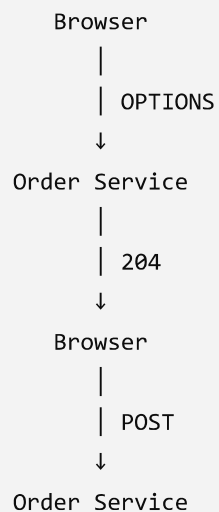
```
OPTIONS /orders
```

```
Origin: https://app.company.com
```

```
Access-Control-Request-Method: POST
```

```
Access-Control-Request-Headers: authorization,content-type
```

Then:



6. Why you may not see OPTIONS in Chrome DevTools

Check these things in **Network**:

Disable cache

Open:

DevTools → Network → Disable cache

Then reload the page.

Also try:

DevTools → Network → Clear

and perform a request again.

Filter by `Fetch/XHR`

Sometimes you're looking at:

Fetch/XHR



and missing what you're expecting because of filtering/grouping.

Search for:

OPTIONS



or use the **Method** column.

7. Another important scenario: API Gateway / Reverse Proxy

In a production architecture you might have:

```
Angular
  |
  ↓
API Gateway
  |
  ├── User Service
  ├── Order Service
  ├── Payment Service
  └── Product Service
```



The browser sees:

Angular → API Gateway



not necessarily:

```
Angular → User Service
Angular → Order Service
Angular → Payment Service
```



So the browser's CORS preflight is handled at the **gateway/origin boundary**.

For example:



The browser doesn't perform CORS preflight between the gateway and internal microservices.

That's why in a microservices architecture you might have **10 services** but see only a few **OPTIONS** requests.

8. The key mental model

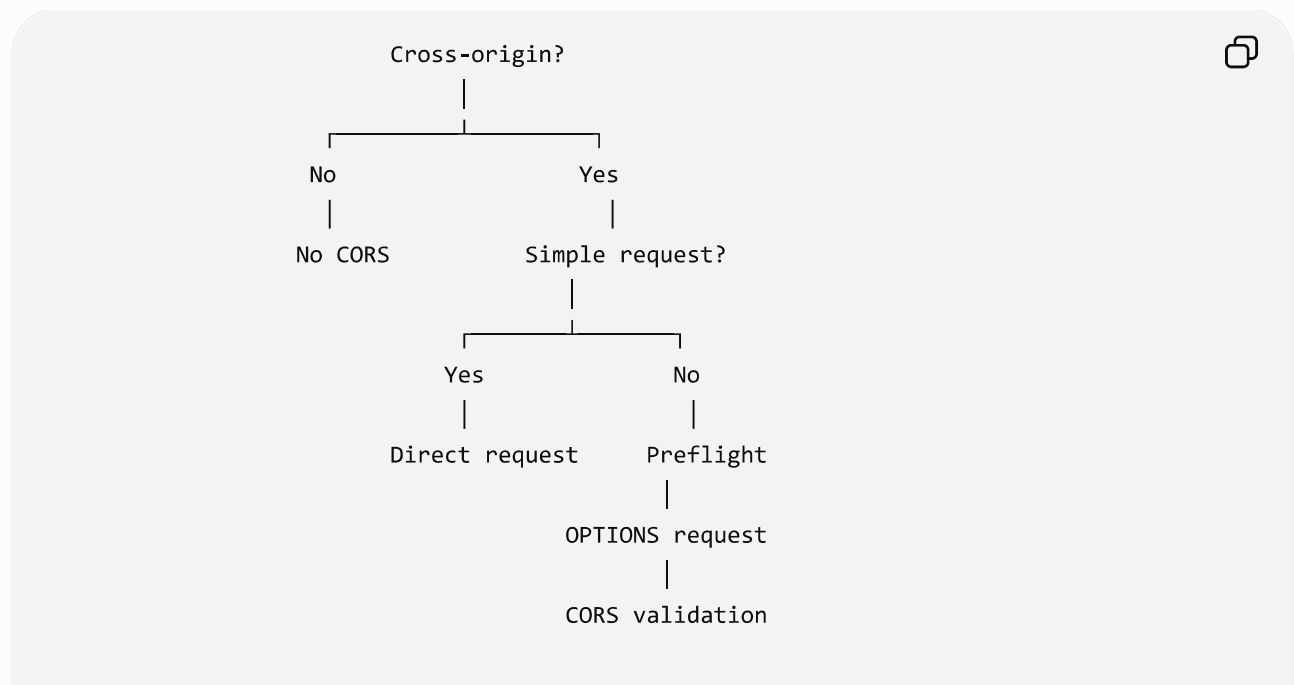
Don't think:

Every API request → OPTIONS

Think:

Cross-origin + request requires preflight + preflight cache expired → OPTIONS

So:



In your Angular + multiple-services scenario

If you have something like:

Angular 19

```
|  
├─ Auth Service  
├─ User Service  
├─ Product Service  
├─ Order Service  
└─ Notification Service
```



I can also explain **exactly when** Chrome will generate OPTIONS for each service, including **Angular HttpClient**, **Authorization headers**, **JWT**, **API Gateway**, **same-origin vs cross-origin**, and **preflight caching**.



Exactly — the important thing is that **preflight** is **determined by the browser based on the request**, not by the microservice itself.

So this:

```
OPTIONS /users  
GET    /users
```




```
GET    /orders

GET    /products

POST   /orders
```

does **not** mean " /users requires CORS but /orders doesn't."

It means the particular /users request met the browser's **preflight conditions**, while the other requests did not.

The easiest way to determine it

For every cross-origin request, ask these 3 questions:

1. Is it cross-origin?

Example:

```
Angular:
https://app.example.com

Users:
https://users.example.com
```



Different origin → **CORS applies**.

But:

```
Angular:
https://app.example.com

API:
https://app.example.com/api/users
```



Same origin → **no CORS/preflight**.

2. Is the request a "simple request"?

A request can avoid preflight if it satisfies the browser's simple-request rules.

Typical simple requests:

```
</> http
```



```
GET /users
```

```
</> http
```



```
POST /orders
```

```
Content-Type: application/x-www-form-urlencoded
```

```
</> http
```



```
POST /orders
```

```
Content-Type: multipart/form-data
```

```
</> http
```



```
POST /orders
```

```
Content-Type: text/plain
```

So:

```
GET /users
```

```
GET /orders
```

```
GET /products
```



can all go directly without `OPTIONS` .

3. Does the request contain something that requires preflight?

This is where Angular applications commonly trigger it.

For example:

```
</> http
```



```
GET /users
```

```
Authorization: Bearer eyJ...
```

`Authorization` is **not** a CORS-safelisted request header, so the browser will generally preflight:

```
OPTIONS /users
```

↓

```
GET /users
```



But imagine:

```
</> http  
  
GET /orders
```



with no non-safelisted headers.

The browser can do:

```
GET /orders
```



directly.

So you could have:

```
/users  
Authorization: Bearer xxx  
    ↓  
OPTIONS  
GET
```



while:

```
/orders  
(no Authorization)  
    ↓  
GET
```



A practical Angular example

Suppose you have:

```
</> TypeScript  
  
this.http.get('https://users.example.com/users');
```



No unusual headers:

```
Browser  
|  
└─ GET /users
```



But your Angular interceptor adds:

</> TypeScript

Authorization: Bearer <token>



Now:

Browser

```
|
|— OPTIONS /users
|
|— GET /users
```



This is why **Angular interceptors** are very important when debugging preflight.

For example:

</> TypeScript

```
intercept(req: HttpRequest<unknown>, next: HttpHandler) {

  const token = this.authService.getToken();

  req = req.clone({
    setHeaders: {
      Authorization: `Bearer ${token}`
    }
  });

  return next.handle(req);
}
```



Now the interceptor potentially changes the CORS behavior of requests.

What about POST?

This is another common misconception:

"POST always causes preflight."

No.

For example:

```
</> http
```



```
POST /orders  
Content-Type: application/json
```

will generally trigger preflight because `application/json` isn't a CORS-safelisted content type.

You'll get:

```
OPTIONS /orders  
POST /orders
```



But:

```
</> http
```



```
POST /orders  
Content-Type: application/x-www-form-urlencoded
```

can be a simple request and therefore may not require preflight.

What about PUT and DELETE?

These methods are not CORS-safelisted.

Therefore, for a cross-origin request:

```
PUT  
PATCH  
DELETE
```



you should generally expect preflight.

For example:

```
DELETE /users/123
```



becomes:

```
OPTIONS /users/123  
DELETE /users/123
```



The interview-friendly rule

You can remember it like this:

Request	Cross-origin?	Usually preflight?
GET	Yes	✗ Not if simple
POST + form-urlencoded	Yes	✗ Not if simple
POST + JSON	Yes	✓
PUT	Yes	✓
PATCH	Yes	✓
DELETE	Yes	✓
GET + Authorization	Yes	✓
GET + custom header	Yes	✓

But: the exact browser decision depends on the full request, including headers, content type, credentials, and other CORS rules.

So how do you know for each microservice?

Don't classify the **service**.

Classify the **HTTP request**.

For example:



Suppose:

User Service

</> http



GET /users
Authorization: Bearer xxx

→ Preflight

OPTIONS /users
GET /users



Order Service

</> http



GET /orders

→ No preflight

GET /orders



Product Service

</> http



GET /products

→ No preflight

GET /products



Order creation

</> http



POST /orders
Content-Type: application/json
Authorization: Bearer xxx

→ Preflight

OPTIONS /orders
POST /orders



So the result might actually look like:

```
OPTIONS /users
GET    /users

GET    /orders

GET    /products

OPTIONS /orders
POST   /orders
```



One more thing: preflight is cached

This is why your DevTools can sometimes be confusing.

Suppose the browser already performed:

```
OPTIONS /orders
```



and the server returned:

```
</> http

Access-Control-Max-Age: 3600
```



For subsequent compatible requests, the browser may use the cached preflight result:

```
POST /orders
POST /orders
POST /orders
```



without showing another `OPTIONS`.

So seeing no `OPTIONS` does not necessarily mean the request doesn't require preflight.

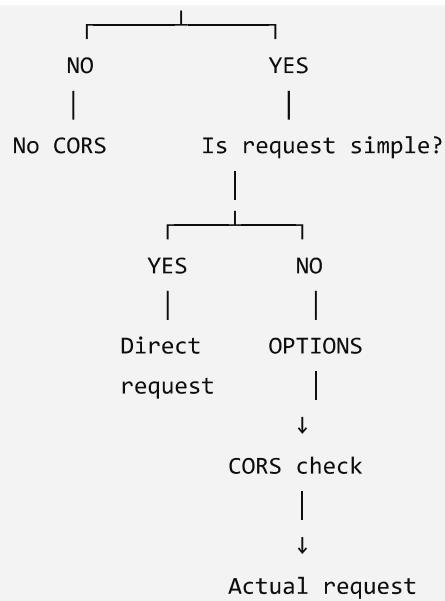
The mental model I'd use in system design

Think:

```
Cross-Origin?
```

```
|
```





And in a microservices architecture, you don't configure "preflight for User Service" or "preflight for Order Service."

You configure **CORS on the server/gateway**, and the **browser decides whether an OPTIONS request is necessary for each cross-origin request**.

If you're using **Angular 19 + API Gateway + multiple FastAPI/Node services**, this distinction becomes especially important because **the browser may only see the gateway as the cross-origin server**, while the gateway communicates with internal services without browser CORS at all.



i was getting the preflight call because of different base url which is not matching with original application url

Yes — **that's the first condition for CORS**, but there's an important distinction:

A different base URL makes the request cross-origin, but it does not automatically mean a preflight (**OPTIONS**) will happen.

Example

Your Angular application:

<https://app.example.com>



API:

```
https://api.example.com/users
```



These are different **origins** because the host is different:

```
app.example.com
  ≠
api.example.com
```



So the browser applies CORS.

But the browser decides whether to send:

```
OPTIONS /users
```



based on the **request characteristics**.

Example 1 — Cross-origin but no preflight

```
</> http
GET https://api.example.com/products
```



If it's a simple request:

```
Angular
  |
  | GET
  ▼
API
```



No `OPTIONS` is necessary.

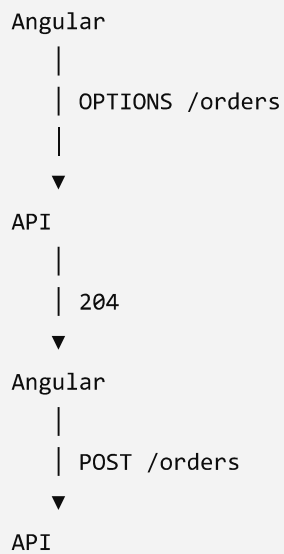
Example 2 — Cross-origin + preflight

If Angular sends:

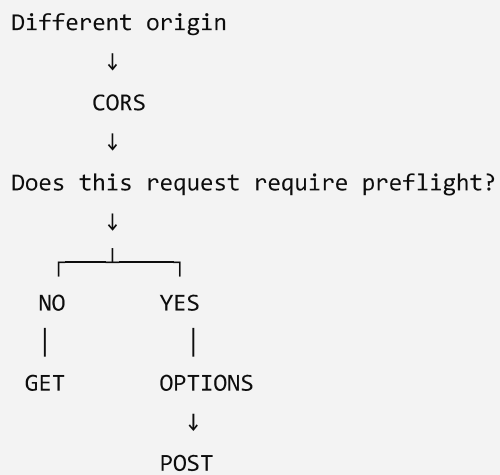
```
</> http
POST https://api.example.com/orders
Authorization: Bearer xxx
Content-Type: application/json
```



the browser needs to check permission first:



So remember these two separate concepts



And different port also counts as a different origin:

http://localhost:4200
http://localhost:8000
↑
different port

So your Angular app calling:

http://localhost:8000/api/users

from:

http://localhost:4200

is cross-origin.

Why your situation probably showed OPTIONS

If your Angular app had:

```
http://localhost:4200
```



and your API base URL was:

```
http://localhost:8080
```



then you had a cross-origin request. If Angular also sent something like:

```
</> http  
  
Authorization: Bearer ...  
Content-Type: application/json
```



the browser would typically do:

```
OPTIONS → http://localhost:8080/users  
GET/POST → http://localhost:8080/users
```



So the different base URL enabled CORS to become relevant; the request itself determined whether preflight was needed.

This distinction is very useful in interviews:

CORS is about origin. Preflight is about whether a cross-origin request needs permission to be sent.



Today 8:26 AM

create an image